

D&D Manager



Giuseppe Coppola

Progetto per Programmazione avanzata - 2025/26

1) Caso d'uso

Dungeons & Dragons è un gioco di ruolo fantasy con vari contenuti ufficiali tra cui oggetti, incantesimi e equipaggiamento.

D&D Manager è un servizio che permette la creazione di incantesimi, oggetti ed equipaggiamenti «homebrew» (personalizzati dall'utente) e la possibilità di visualizzare in un database sia i contenuti ufficiali che i contenuti creati dall'utente.

I dati ufficiali sono stati reperiti da: [D&D 5e SRD API](#)

2) Applicazione

L'applicazione si avvia dalla homepage, da dove è possibile accedere alle varie funzioni tramite un menù a tendina:

- Inizializzazione del database del servizio
- Creazione di nuovi contenuti
- Visualizzazione dei dati nel database

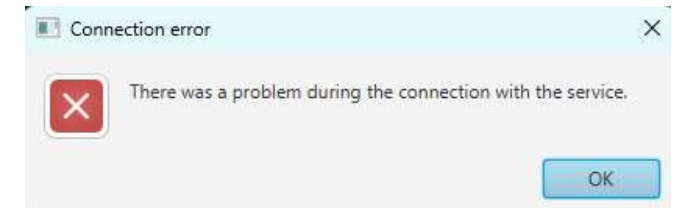
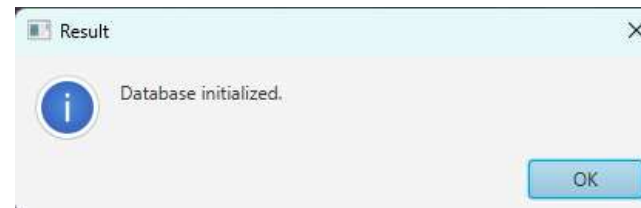
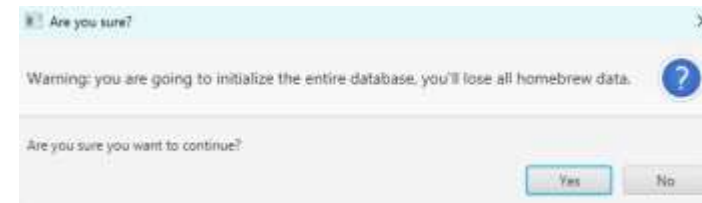
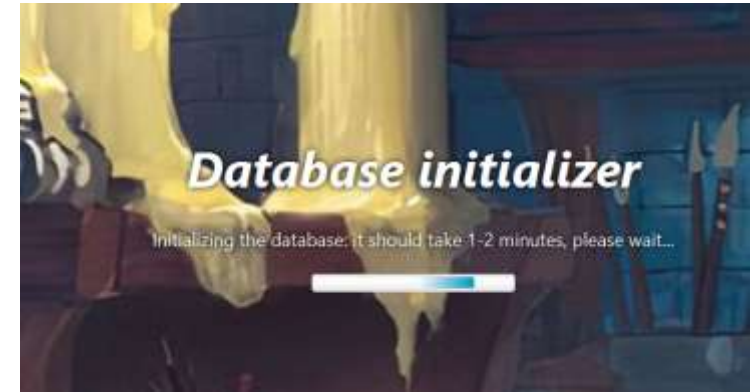
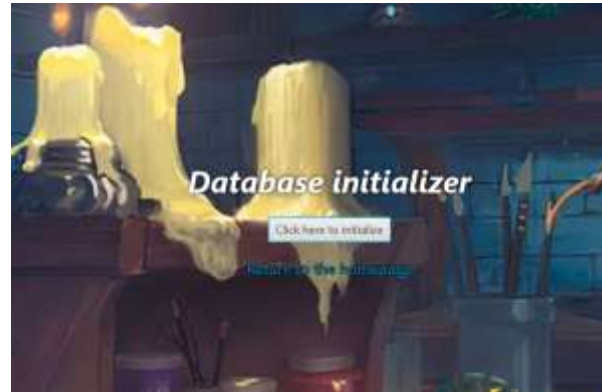
Ogni pagina contiene un link per ritornare alla homepage.



2.1) Inizializzazione del database del servizio

Questa pagina consente l'inizializzazione del database da parte dell'utente: rimuove tutti i dati, sia ufficiali che «homebrew» e successivamente ripopola il database con i dati ufficiali presi dall'API.

Richiede una riconferma dopo aver premuto il pulsante e dopo una breve attesa apparirà un prompt che comunica l'esito dell'operazione.



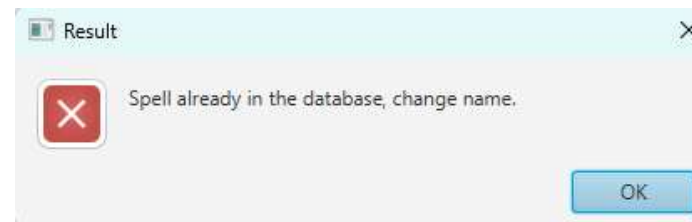
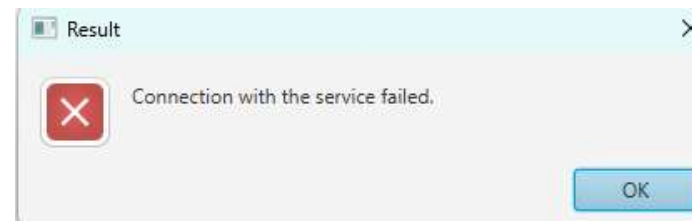
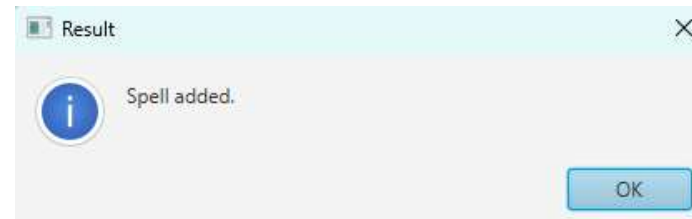
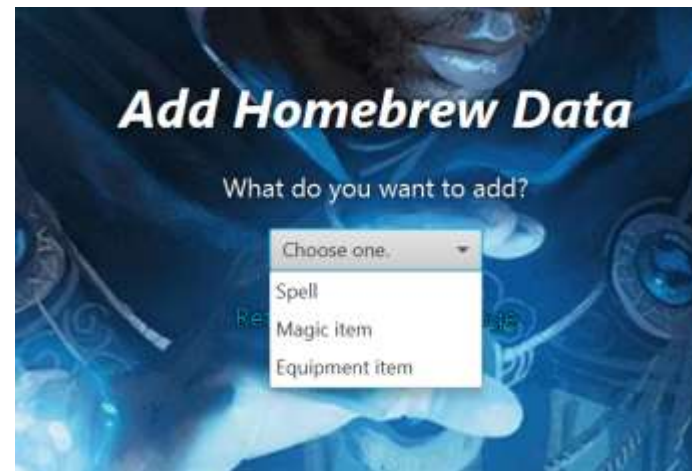
2.2) Creazione di nuovi contenuti

Permette l'aggiunta di incantesimi, oggetti ed equipaggiamenti al database. La categoria da inserire è selezionabile dal menù a tendina. Dopo aver compilato i campi obbligatori premendo il pulsante in basso è possibile aggiungere il contenuto al database, se questo non è già presente.

L'esito dell'aggiunta viene mostrato tramite un messaggio pop-up.

Per gli oggetti magici è possibile aggiungere anche un'immagine di massimo 16MB.

I campi obbligatori sono contrassegnati da un asterisco (*) e verranno segnati di rosso se mancanti al tentativo di aggiunta.



Add Homebrew Data

What do you want to add?

Spell

Name (*)

Spell test

Spell range (*)

30m

Level

1

Description (*)

Effects at higher levels

Material

Components (*)

☒ Somatic

☒ Material

☐ Verbal

☐ Ritual

☒ Yes

☐ Concentration

☐ Yes

Duration (*)

10 minutes

Casting time (*)

Attack type

Spell school

Type

Area of effect

Range

Add spell

[Return to the homepage](#)

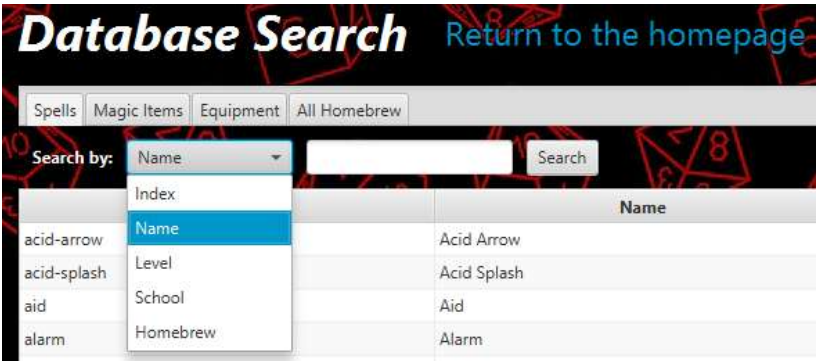
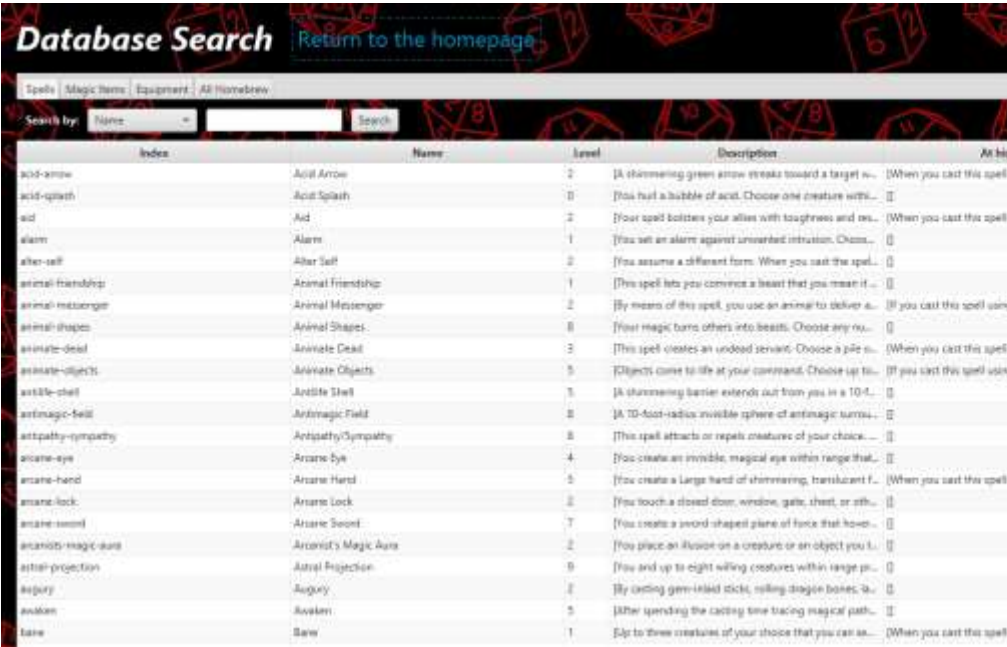
2.3) Visualizzazione dei dati nel db

Mostra il contenuto del database del servizio, con una pagina per ogni tipo di contenuto e una esclusiva per i contenuti «homebrew».

Presenta anche una funzione di ricerca selezionando il criterio e inserendo cosa cercare.

Tramite tasto destro del mouse su un elemento si apre un menu che permette l'eliminazione del contenuto selezionato dal database.

Tramite doppio click su un elemento appare un pop-up che mostra il contenuto ordinato dell'elemento selezionato.



2.4) Note tecniche dell'applicazione

Gli oggetti creati permettono l'aggiunta di una immagine e per gestirla tramite JSON viene convertita in Stringa BASE64 dall'applicazione e poi viene riconvertita in byte[] dal servizio.

Le operazioni che interagiscono con il servizio sono tutte eseguite in threads per non bloccare l'interfaccia grafica e alcune interazioni vengono bloccate durante l'operazione.

3) Servizio

Il servizio è realizzato tramite Spring e si occupa della connessione col database MySQL e dell'esecuzione delle query.

In seguito la lista delle API offerte all'applicazione:

- GET db/initialize Inizializza l'intero database.
- GET db/all Restituisce tutto il contenuto del db in formato JSON
- GET db/official Restituisce il contenuto ufficiale in formato JSON
- GET db/homebrew Restituisce il contenuto "homebrew" in formato JSON
- GET db/delete/homebrew Rimuove tutti i contenuti "homebrew" dal db
- GET db/delete/official Rimuove tutti i contenuti ufficiali dal db

- GET db/*/filter/all Restituisce tutti gli elementi del tipo *
- GET db/*/filter/homebrew Restituisce gli elementi "homebrew" del tipo *
- GET db/*/filter/official Restituisce gli elementi ufficiali del tipo *
- POST db/*/add Aggiunge il dato * ricevuto al database
- POST db/*/delete Rimuove il dato * ricevuto dal database
- GET db/*/search/{name} Restituisce il dato * che ha il nome ricevuto
- GET db/*/index Restituisce il dato * che ha l'indice ricevuto

* : *spells* per gli incantesimi, *magic_items* per gli oggetti, *equipment_items* per gli equipaggiamenti

3.1)Note tecniche del servizio

Sono presenti due javabeans riferiti agli oggetti, MagicItem e MagicItemEncoded, dove il secondo è il tipo di dato che il servizio si aspetta dall'applicazione, con l'immagine in Stringa BASE64, che poi viene convertito in MagicItem con l'immagine in byte[].

La classe ComponentListJsonConverter è usata per convertire il tipo di dato List<String> ricevuto dall'applicazione in JSON automaticamente da Spring per poterlo inserire nel database.

Il database viene creato all'avvio da Spring usando il file «schema.sql» presente in «\src\main\resources». Invece lo script «createDatabase.sql» presente nella root directory del progetto serve per crearlo manualmente.

4) Unit test

L'applicazione è dotata di una unit test che va a verificare le funzionalità base:

- Inizializzazione del database
- Aggiunta di dati nel database
- Rimozione di dati dal database

I test vengono eseguiti in un ordine stabilito tramite:

- `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)`
- `@Order()`

```
T E S T S
-----

Running it.unipi.progetto_applicazione.ApplicationTest
TEST INITIALIZE THE DATABASE --- START
Starting the database initialize...
TEST INITIALIZE THE DATABASE --- SUCCESS

TEST ADD DATA IN THE DATABASE --- START
Adding the test spell...|
Adding the test magic item...
Adding the test equipment item...
TEST ADD DATA IN THE DATABASE --- SUCCESS

TEST REMOVE DATA FROM THE DATABASE --- START
Removing the test spell...
Removing the test magic item...
Removing the test equipment item...
TEST REMOVE DATA FROM THE DATABASE --- SUCCESS

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time el

Results:

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
```

5)Utilizzo di AI

Durante l'implementazione del progetto è stato usato [Gemini 3 Pro](#) per il debugging e per la spiegazione e corretta implementazione di classi di Java utili.

In particolare la conversione da `List<String>` a JSON, la conversione da stringa Base64 a `byte[]`, la gestione di threads che interagiscono con l'interfaccia dell'applicazione e l'uso di executors invece che threads con Spring.